Introduction:

In the Phase I research, I looked at a popular streaming platform, Netflix, which uses different internet communication methods and performance tools that help it be safe and reliable. Like my project, Netflix uses HTTP over TCP through port 433 for their communication flow and uses security mechanisms like TLS to encrypt the information of customers and other confidential data. Netflix holds three HTTPS links for each video file and these files are stored on three different CDN servers, the app then makes several TCP connections to these servers and downloads pieces of the file using normal secure HTTP.

For my prototype, I plan to create an E-Commerce shop that allows users to be able to log in, browse a list of products, add items to their cart, and complete a checkout process using HTTP secured communication. The system will also be able to send responses, such as “payment successful” from the server to the client. In addition, I also planned to develop a front-end with HTML and JavaScript, which will enable the client to perform all these actions interactively. Meanwhile, the back end of the system will be created using Python, which will handle the server-side operations such as user authentication, product management, and order processing.

System Design:

For my E-Commerce prototype, the communication between the client and the server will follow a Client/Server (C/S) model using the port and protocol HTTP on port 5000. The front-end acts as the client and the back-end Python server manages requests and responses while serving as the API server. While all communication occurs over HTTP on port 5000, the back end will also allow secure login protocols, including session management. When a user logs in successfully, the server uses JSON web tokens to create essentially a code using the HS256 algorithm. This token has a 30 minute expiration time, ensuring that access is temporary and secure. When using the front end, this token must be used for when the user checks out which confirms that the user has permission to perform the checkout.

Implementation Details of the Prototype:

The front end will be built using HTML, JavaScript, and CSS which handles the user interaction and sends the API requests. The back end will use Python, with libraries such as Flask used to create the web server and JWT which generates and verifies the JSON web tokens. The server was run locally through terminal and when testing this server Postman was also used. URLs endings included

- /login

- /products
- /cart
- /checkout

In Postman, tokens were sent using the “Authorization: Bearer <token> header.

In order for a successful login the correct username and password must be entered, and after this the server returns a valid token and you will be granted a button which takes the user to the home page of the e-commerce shop as well as an output saying “Login successful! Token saved to localStorage.” If it is an invalid login, the server will not send a token. Interactive add to cart buttons are placed which sends the item to the users cart. At checkout the user must insert the token and after checkout they receive an output saying “Payment successful.”

Performance Technique:

Although my prototype is small, in a real world scenario my front end could be delivered through a CDN to improve speed and global access, similar to Netflix which I did research on in Phase I. The JWT tokens could be managed with shorter expiration times, and the whole project could be deployed to the cloud so it can handle more users smoothly.

Conclusion:

This project allowed me to design an E-Commerce prototype with secure communication and interactive features. One of the main challenges was getting the front end and back end to work smoothly together, especially when sending and receiving data between the browser and the server. It also took time to fully understand how each part of the system communicates, from authentication to API calls. Through this project, I learned how to implement HTTPS communication, user authentication, and server side handling in Python, as well as front end interactivity with JavaScript. In the future, I would like to increase performance by working with CDN deployment, using caching and compression, as well as include more complex features.